RESTAURANT TABLE RESERVATION SYSTEM

??????????????????????????????

PROJECT DOCUMENTATION

PROJECT DOCUMENTATION
Table & Thyme | Python Full-Stack Individual Project

DJANGO ? MYSQL ? BOOTSTRAP

1. Project Title and Introduction

1.1 Project title

Restaurant Table Reservation System — a web application branded as **Table & Thyme**.

1.2 Introduction

The Restaurant Table Reservation System gives restaurant customers an online way to request and manage table bookings. Customers can register, sign in, create reservations, and view or update their own bookings. The application provides a dashboard for a quick summary and lets customers search and filter their reservation list.

Many restaurants still accept bookings through phone calls, paper lists, or scattered messages. A centralized application makes reservation details easier to find and helps reduce manual record handling. The system is intended as an academic full-stack project and a foundation for a restaurant booking workflow; it does not include live table inventory, payment processing, or customer notifications.

The application follows Django's model-view-template architecture. Django handles authentication, URL routing, request processing, validation, and database access through its ORM. The browser interface uses Django templates, HTML, CSS, Bootstrap, and JavaScript. MySQL stores application records in a configured deployment.

2. Project Objectives

- Provide customers with an online method to request a restaurant table.
- Let customers review, edit, and delete their own reservations.
- Keep reservation records in a relational database linked to customer accounts.
- Make bookings easier to retrieve with name/phone search and date/status filters.
- Show reservation totals and recent bookings on a customer dashboard.
- Validate submitted data in the browser and again on the server.
- Prevent a non-cancelled reservation from using a table already booked for the same date and time.
- Demonstrate a maintainable full-stack application using Python, Django, MySQL, and browser technologies.

3. Technologies Used

Layer	Technology	Use in the project
Programming language	Python 3.10+	Application and server-side validation
Backend framework	Django 5.2	URL routing, views, forms, authentication, ORM, and administration
Frontend	HTML5 and Django templates	Page structure and server-rendered screens
Styling	CSS3 and Bootstrap 5.0.2	Responsive layout, components, and page styling
Client-side scripting	JavaScript	Quick reservation-form checks before submission
Database	MySQL 8+	Persistent user and reservation data
Database adapter	PyMySQL	Django-to-MySQL connectivity
Configuration	`python-dotenv`	Loads local settings from `.env`

Development tools	VS Code, Git, MySQL Workbench	Editing, version control, and database inspection

Bootstrap assets are loaded from a CDN, so internet access is required for those assets unless the project is configured to serve them locally.

4. System Requirements

4.1 Hardware

Component	Requirement
Computer	A system capable of running Python and a local MySQL server
Memory	4 GB RAM or more recommended for local development
Storage	Free space for the project, Python environment, and database
Display	1366 × 768 or higher recommended for comfortable development

4.2 Software

Component	Requirement
Operating system	Windows 10/11, macOS, or Linux
Python	Python 3.10 or later
Framework	Django 5.2 (installed from `requirements.txt`)
Database	MySQL Server 8+ for the configured application database
Browser	Current Chrome, Firefox, Edge, or another modern browser
Editor	VS Code or another Python-capable code editor
Optional database tool	MySQL Workbench

5. System Design

5.1 Application architecture

The browser sends HTTP requests to Django. URL patterns route requests to views; views apply authentication and business rules, use forms for input validation, and query or update models through the Django ORM. Django renders templates into HTML responses. The configured database stores user and reservation data.

Request flow: Browser → Django URL configuration → view/form → Django ORM → MySQL → rendered template → Browser.

5.2 Main user workflow

1. A visitor creates an account or signs in.
2. The authenticated customer opens the dashboard or reservation list.
3. The customer submits a booking with contact details, date, time, guest count, and table number.
4. The server validates the submission and saves it under the signed-in customer.
5. The customer can search/filter their bookings and edit or delete a booking they own.

6. An authorized administrator can inspect reservation records through Django admin.

5.3 Access control and validation

- Django authentication manages user passwords and sessions.
- The dashboard and reservation pages require an authenticated session.
- Reservation updates and deletions look up records by both reservation ID and the current customer, so another customer's record is not returned by those actions.
- Reservation forms reject past dates, phone numbers containing fewer than 10 or more than 15 digits, guest counts outside 1–20, and table numbers outside 1–50.
- The reservation form checks for another non-cancelled booking for the same table, date, and time. A cancelled booking does not block a new booking. This is application-level form validation, not a database uniqueness constraint.
- Django forms and CSRF tokens provide server-side validation and request protection; JavaScript provides immediate client-side checks but is not a substitute for server validation.

6. Database Design

6.1 Entity relationship

```
auth_user (1) ──────< (many) reservations_reservation
```

The application uses Django's built-in user/authentication tables and the custom `reservations_reservation` table. One user may own many reservations; each reservation belongs to one user. Deleting a user cascades to that user's reservations.

6.2 Reservation table

Field	Django/model type	Description
`id`	BigAutoField, primary key	Unique reservation identifier
`customer_id`	Foreign key to the configured user model	Account that owns the reservation; cascade deletion
`customer_name`	CharField(100)	Name supplied for the booking
`phone`	CharField(15)	Contact phone number
`reservation_date`	DateField	Requested booking date
`reservation_time`	TimeField	Requested booking time
`guests`	PositiveSmallIntegerField	Guest count from 1 through 20
`table_number`	PositiveSmallIntegerField	Table number from 1 through 50
`special_request`	TextField, optional	Optional customer note
`status`	CharField(20)	Pending, Confirmed, Cancelled, or Completed; defaults to Pending
`created_at`	DateTimeField	Automatically recorded creation timestamp

The model orders reservation records by descending date and time. The table/date/time conflict check is performed by the reservation form and excludes cancelled bookings and the current record during an edit.

7. Module Description

Module	Description
Registration	Uses Django's `UserCreationForm`; a successful registration signs the customer in.
Login and logout	Uses Django authentication views and session handling. Logout is submitted as a POST request.
Dashboard	Shows the signed-in customer's total, pending, and confirmed reservation counts and up to five recent bookings.
Reservation creation	Presents a form for customer name, phone, date, time, guests, table number, and an optional request.
Reservation list	Shows the signed-in customer's bookings and supports search by name or phone plus date and status filters.
Reservation update	Allows the customer to edit a reservation they own, subject to the same form validation.
Reservation deletion	Displays a confirmation screen before deleting the customer's booking.
Validation	JavaScript provides quick checks; Django validates all submitted data on the server, including duplicate active table/time requests.
Administration	Django admin lists reservations and provides status/date filters and name/phone search for authorized administrators.
Shared interface	Django templates and responsive CSS/Bootstrap provide a consistent navigation bar, messages, and page layout.

7.1 Main URL routes

Route	Purpose
`/`	Customer dashboard (login required)
`/register/`	Customer registration
`/login/`	Customer login
`/logout/`	Logout (POST)
`/reservations/`	Reservation list, search, and filters
`/reservations/add/`	Create a reservation
`/reservations/<id>/edit/`	Update an owned reservation
`/reservations/<id>/delete/`	Confirm and delete an owned reservation
`/admin/`	Django administration

8. Installation and Execution

The following steps describe local development on Windows PowerShell. Use credentials of your own; do not commit `.env` or production secrets.

7. Install Python 3.10 or later and MySQL Server 8 or later.
8. Open the project directory in PowerShell and create a virtual environment:

```
py -m venv .venv
.\.venv\Scripts\Activate.ps1
```

9. Install project dependencies:

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

10. In MySQL, create a database and a dedicated local application user. Replace the example password with a strong local password:

```
CREATE DATABASE restaurant_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'restaurant_app'@'localhost' IDENTIFIED BY 'replace-with-a-strong-password';
GRANT ALL PRIVILEGES ON restaurant_db.* TO 'restaurant_app'@'localhost';
```

11. Create the local environment file and set `DJANGO_SECRET_KEY`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DB_HOST`, and `DB_PORT` as shown in `.env.example`:

```
Copy-Item .env.example .env
```

12. Apply the migrations included with the project and create an administrator account:

```
python manage.py migrate
python manage.py createsuperuser
```

13. Start the development server and open the application:

```
python manage.py runserver
```

Customer site: `http://127.0.0.1:8000/`

Django admin: `http://127.0.0.1:8000/admin/`

9. Testing

The reservation workflow tests were run while preparing this documentation:

```
python manage.py test reservations --settings=restaurant_project.test_settings
```

Result: 6 tests passed. The test settings use an isolated in-memory SQLite database, so this test command does not require the configured MySQL server.

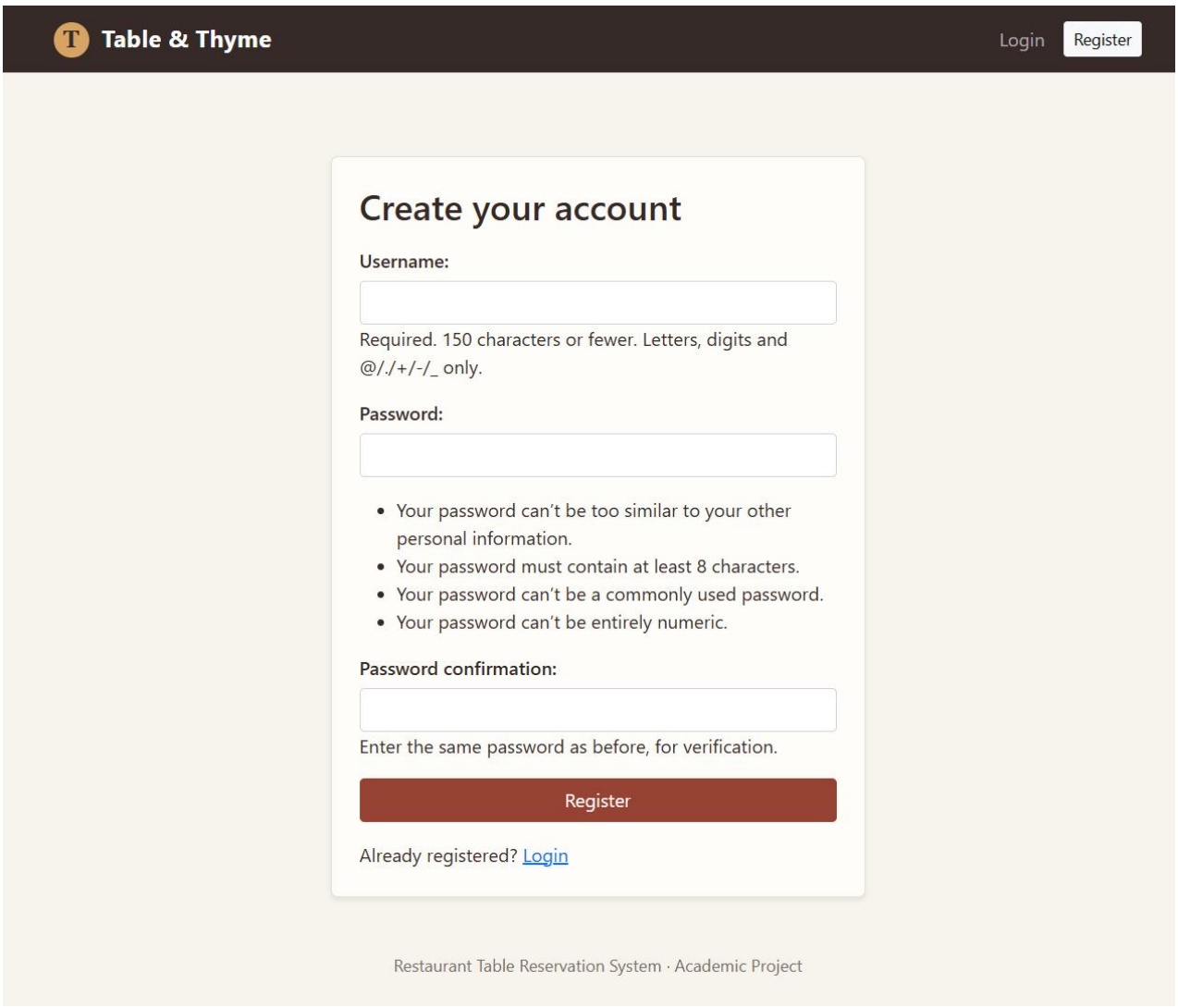
Test area	Verified behavior
Authentication	Registration, login, logout, and redirect from a protected dashboard
Reservation lifecycle	Create, update, list, and delete a reservation
Dashboard	Reservation count is updated after a booking is created
Search and filters	Search by customer name or phone with date and status filters
Input validation	Past dates, invalid phone numbers, and guest counts above the allowed range are rejected
Table availability	Duplicate active table/date/time bookings are rejected; cancelled bookings do not block a new booking

Ownership	A customer receives not-found responses when attempting to edit or delete another customer's reservation

10. Application Screenshots

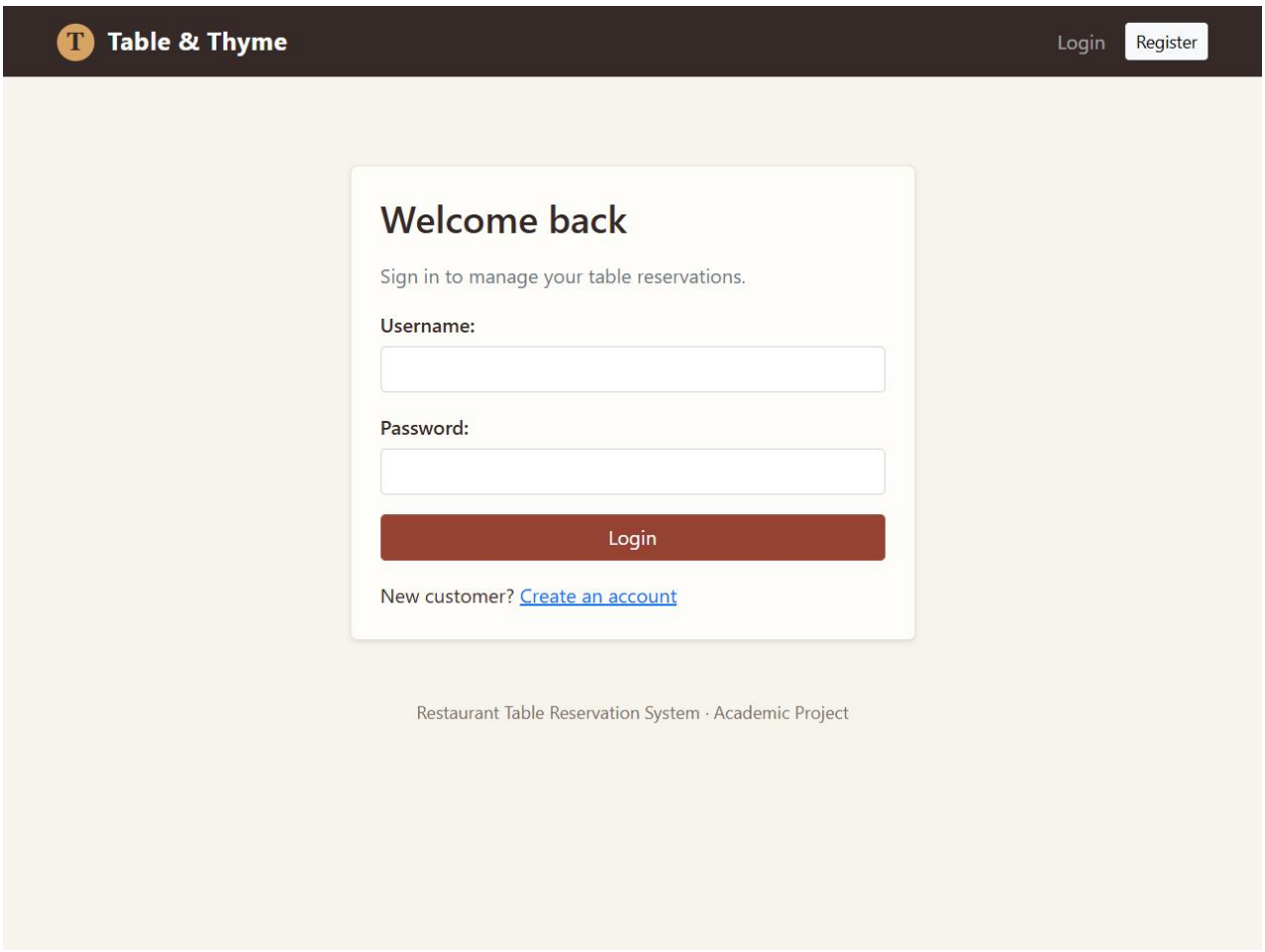
The following screenshots are included from the project's `screenshots` directory. Reservation and database screens use sample records for demonstration.

Figure 1. Customer registration



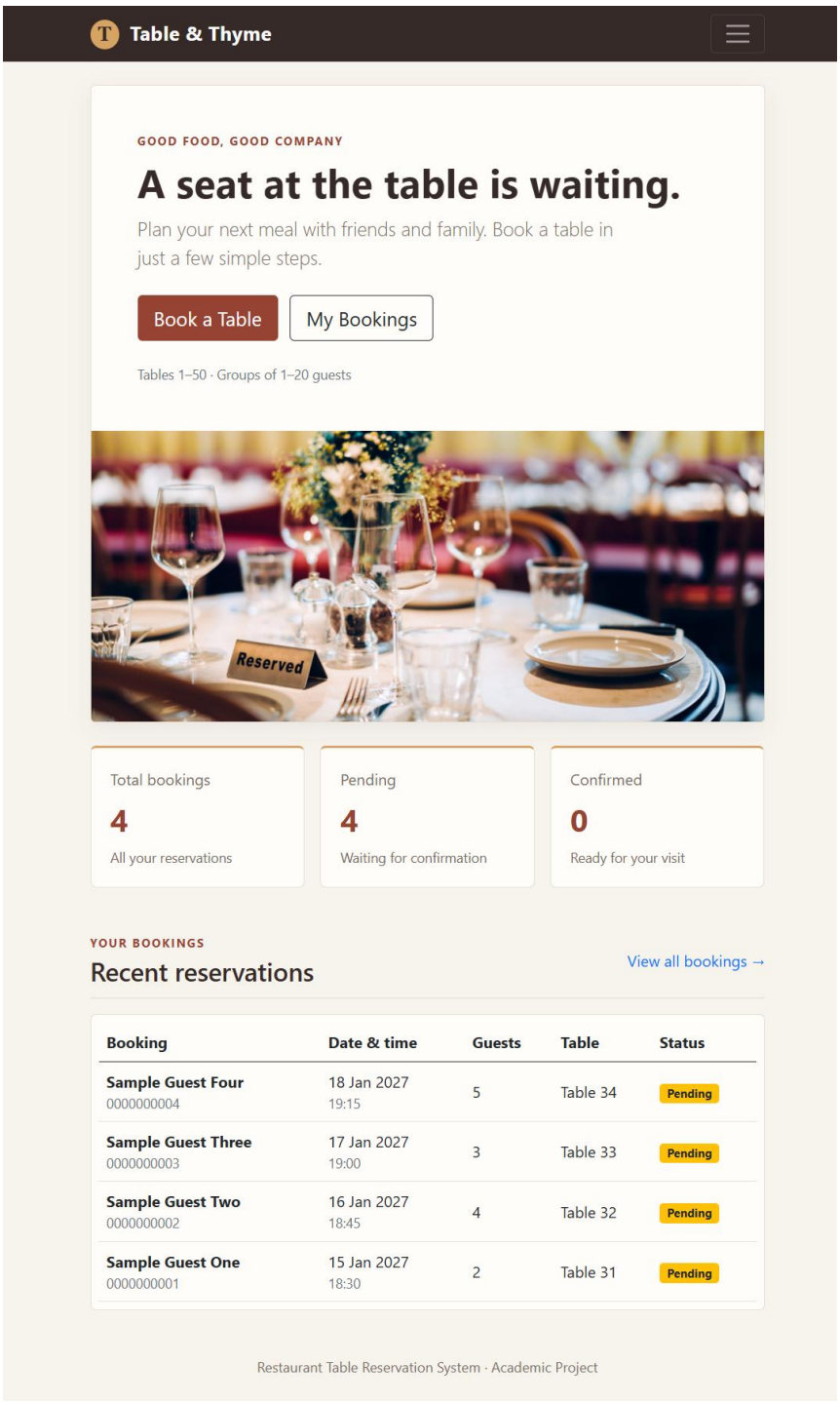
The registration screen creates a customer account using Django's built-in user creation form.

Figure 2. Customer login



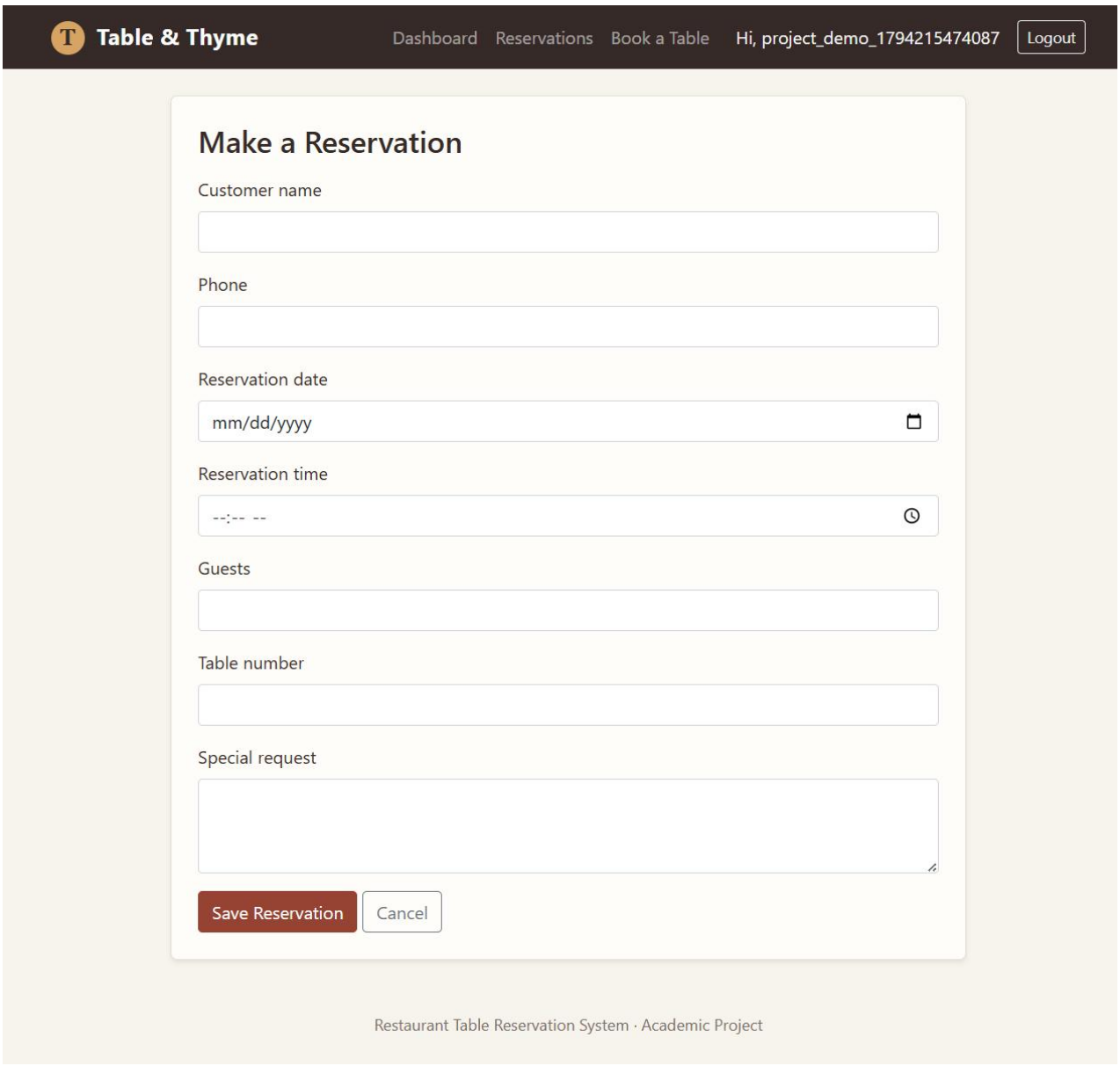
The login screen authenticates an existing customer.

Figure 3. Customer dashboard



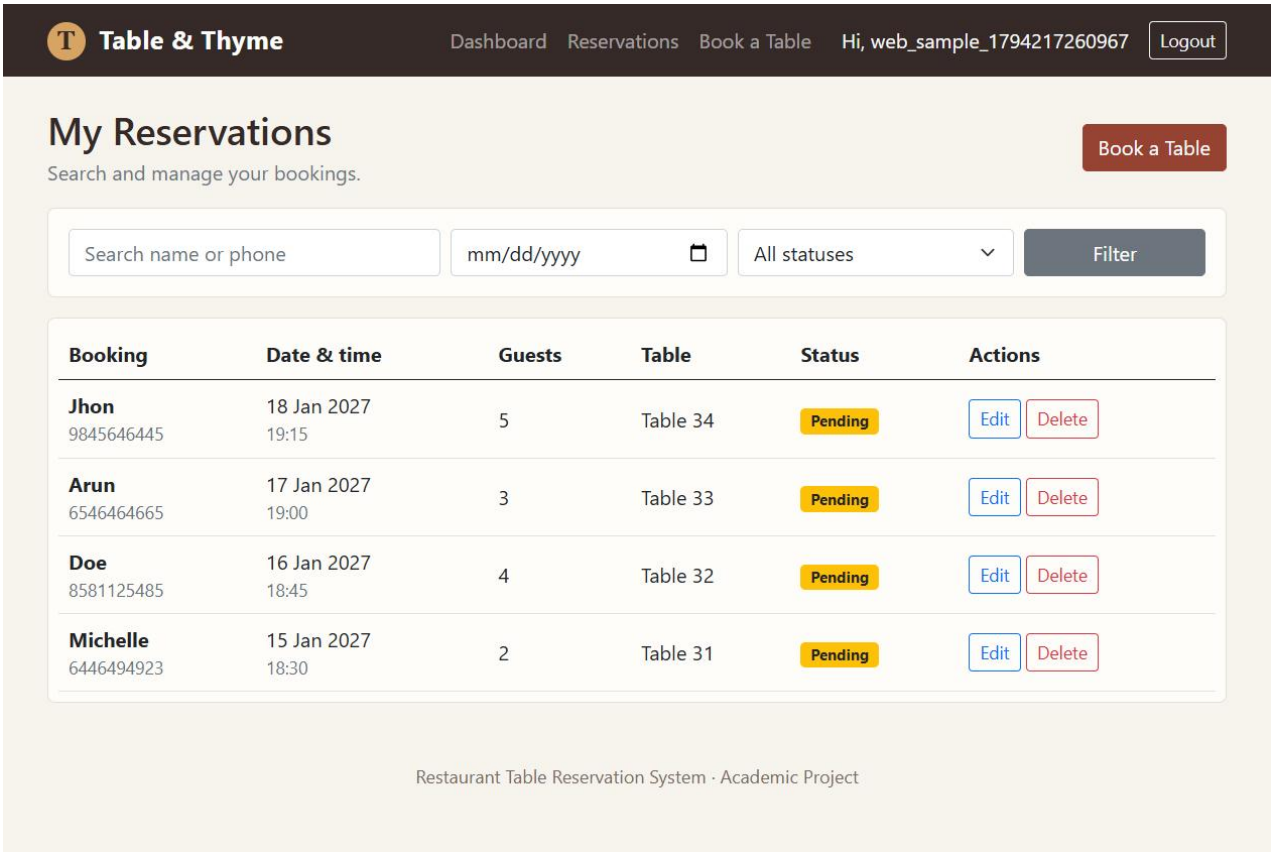
The dashboard displays booking totals, recent reservations, and shortcuts to the main customer actions.

Figure 4. Create reservation



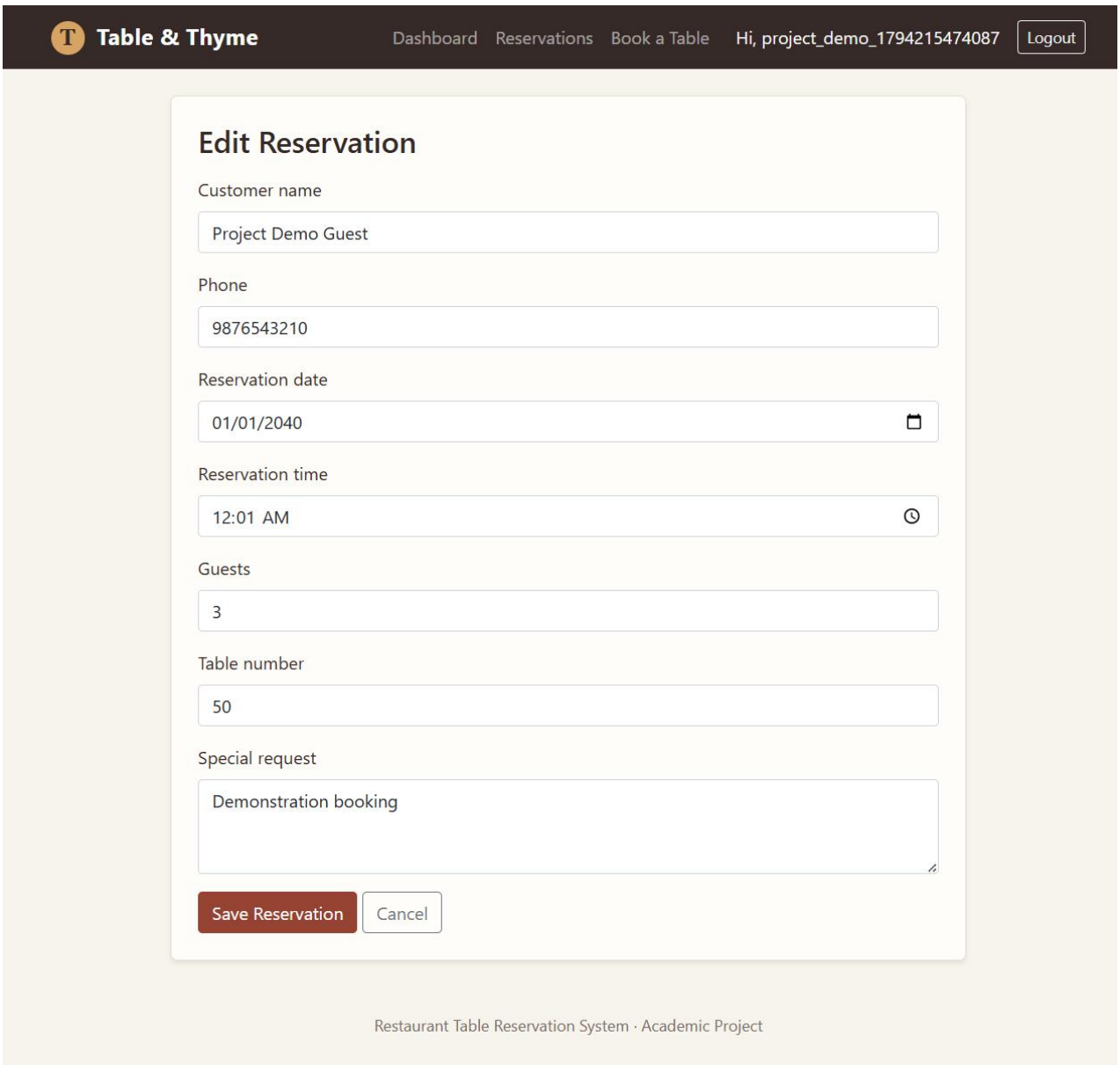
Customers enter booking contact information, date and time, guest count, table number, and an optional request.

Figure 5. Reservation list, search, and filters



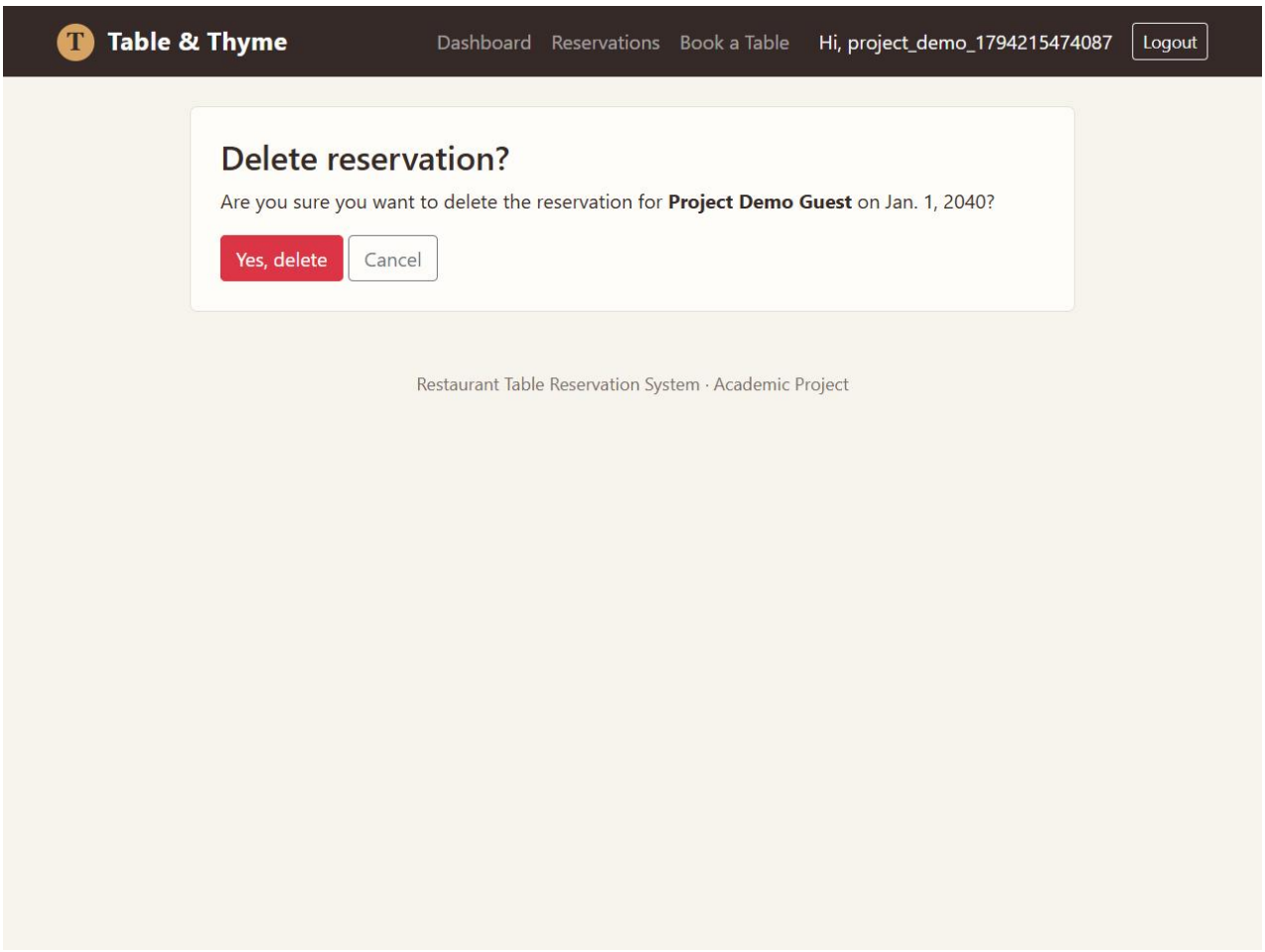
The list presents customer bookings with controls to search by name or phone and filter by date and status.

Figure 6. Edit reservation



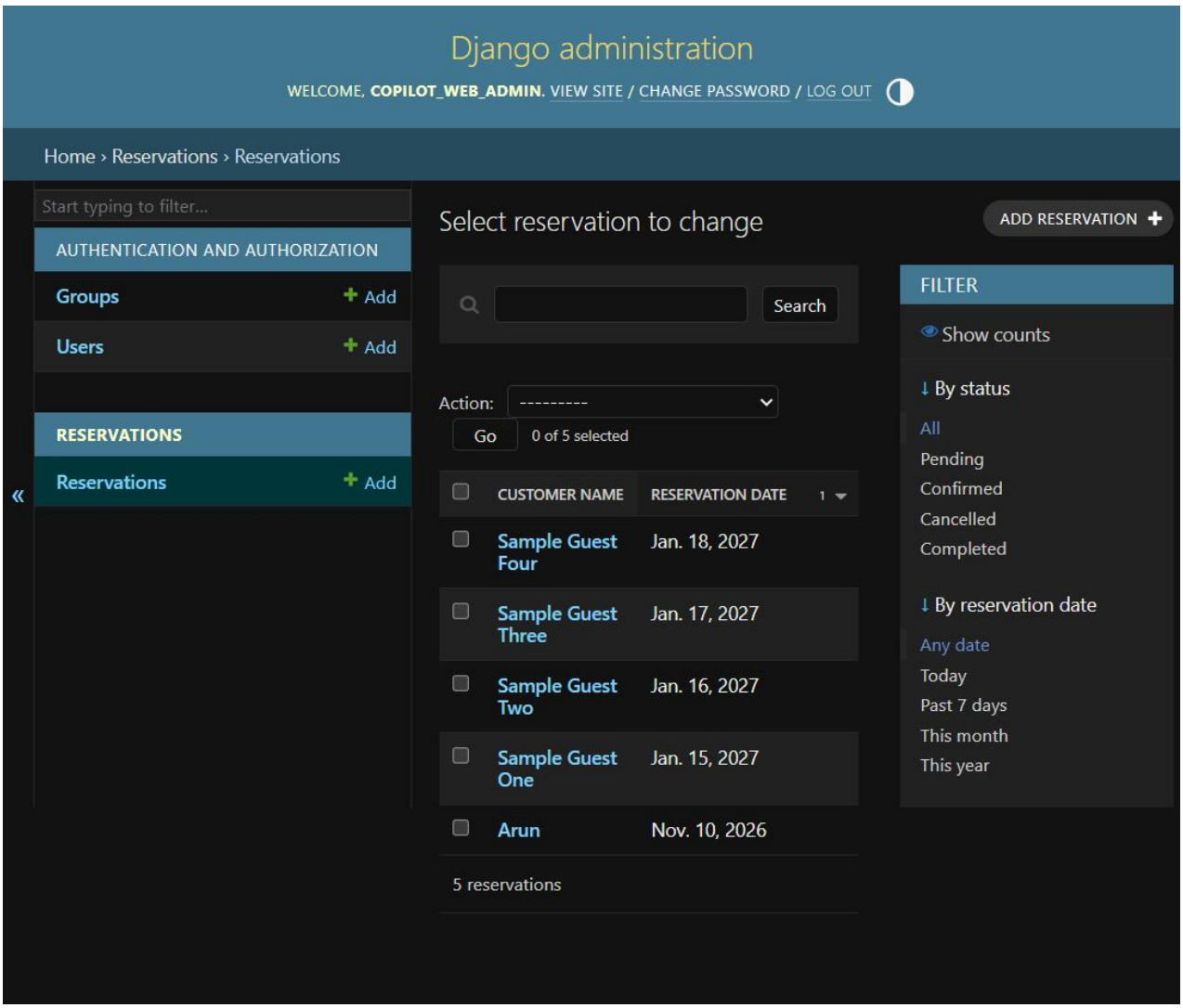
The edit screen loads an existing reservation into the form for an authorized customer.

Figure 7. Delete confirmation



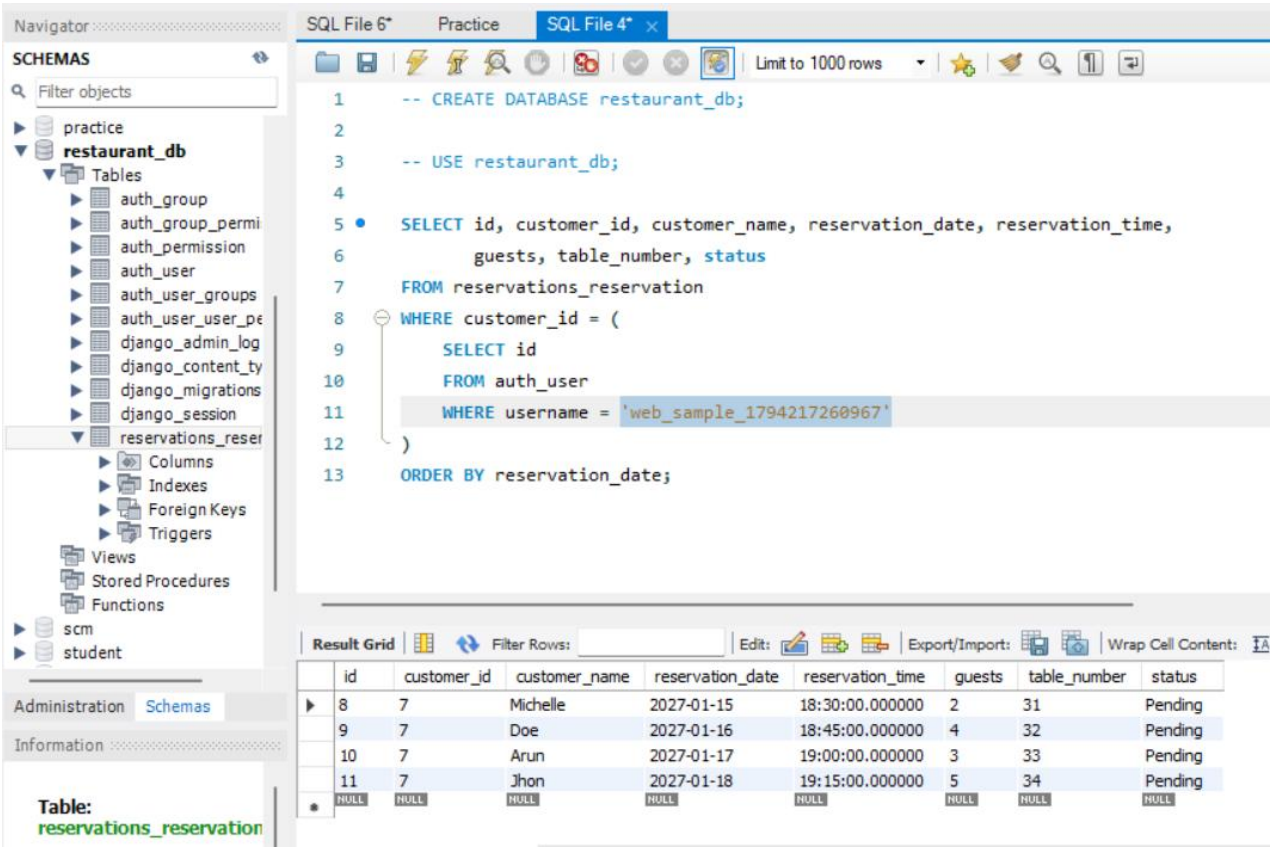
The confirmation screen asks the customer to confirm before a booking is deleted.

Figure 8. Django admin reservations



The administration screen provides a staff-facing view of reservation records.

Figure 9. MySQL database view



MySQL Workbench shows the reservation table and sample records stored in the configured database.

11. Conclusion

The Restaurant Table Reservation System demonstrates an end-to-end Django application for customer authentication and table booking management. It combines relational data modeling, server-rendered pages, form validation, per-customer access controls, reservation search and filtering, and an administrative interface. The workflow tests pass against an isolated test database, and the included screenshots document the main customer, administrator, and database screens.

The current version is a project foundation rather than a live restaurant availability service. It represents tables by numbers 1–50 and accepts a requested time; it does not maintain a real table inventory or opening-hours schedule.

12. Future Enhancements

- Add a managed table inventory and a real-time availability calendar.
- Configure restaurant opening hours, booking durations, and capacity rules.
- Send email or SMS reservation confirmations and reminders.
- Add an administrator workflow for confirming or cancelling bookings.
- Add online payment support if required by the restaurant.
- Add reporting, export, and reservation history features.

Project Repository

Source code, migrations, tests, and screenshots: Restaurant Table Reservation System on GitHub.